

Software Requirements and Design Document

For

Group 3

Version 2.0

Authors:

Tyler Drake

Christian Wilsey

Luis Villanueva

Emmanuel Reyes Martinez

Jamie Velasquez

1. Overview (5 points)

Give a general overview of the system in 1-2 paragraphs (similar to the one in the project proposal).

Spear-It-Skins is a web-based CS2 skin market aggregation system that allows users to browse, search, and filter CS2 item listings based on current market data. The system collects item pricing and listing information from the Skinport API, normalizes the data, stores it in PostgreSQL, and exposes it through a backend REST API for frontend consumption.

The system is designed to help users quickly compare market data for CS2 skins, including price, quantity, weapon type, exterior condition, StatTrak status, and item image information. The backend refreshes market data on a scheduled interval so users can access recently updated pricing without requiring the frontend to query third-party APIs directly.

2. Functional Requirements (10 points)

*List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just what). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.*

FR-1: High Priority

The system shall retrieve CS2 item market data from the Skinport API.

FR-2: High Priority

The system shall normalize raw Skinport item data into a consistent internal item format.

FR-3: High Priority

The system shall store stable item metadata, current market state, and historical price snapshots in a PostgreSQL database.

FR-4: High Priority

The system shall allow users to retrieve a paginated list of CS2 items through a REST API endpoint.

FR-5: High Priority

The system shall allow users to search items by item name.

FR-6: High Priority

The system shall allow users to filter items by weapon type, exterior condition, StatTrak status, minimum price, and maximum price.

FR-7: Medium Priority

The system shall return item information including market hash name, weapon, exterior, image URL, minimum price, suggested price, quantity, and last updated timestamp.

FR-8: Medium Priority

The system shall expose a health endpoint that reports whether the backend service is running.

FR-9: Medium Priority

The system shall expose a database health endpoint that reports whether the backend can connect to PostgreSQL.

FR-10: Medium Priority

The system shall refresh item market data automatically on a scheduled interval.

FR-11: Low Priority

The system shall cache recently fetched item data in memory to support health reporting and reduce reliance on repeated refreshes.

3. Non-functional Requirements (10 points)

List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.

NFR-1: Performance

The system should return paginated item results without sending the full item dataset in a single API response.

NFR-2: Reliability

The system should prevent overlapping refresh jobs from running at the same time.

NFR-3: Data Integrity

The system should avoid duplicate logical items by using each item's market hash name as a unique identifier.

NFR-4: Recoverability

The system should roll back database changes if an error occurs during a refresh transaction.

NFR-5: Maintainability

The backend should separate routing, database access, API fetching, scheduled jobs, and application setup into separate modules.

NFR-6: Usability

The API response format should be simple enough for the frontend to consume directly.

NFR-7: Portability

The backend should run locally using Node.js and should be deployable to a cloud hosting provider with environment variables.

NFR-8: Security

Database credentials should be stored in environment variables rather than hardcoded into the source code.

NFR-9: Scalability

The database should use indexes to improve searching and lookup performance as the number of items grows.

4. Use Case Diagram (10 points)

This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.

Textual descriptions of use cases: For the first increment, the textual descriptions for the use cases are not required. However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.

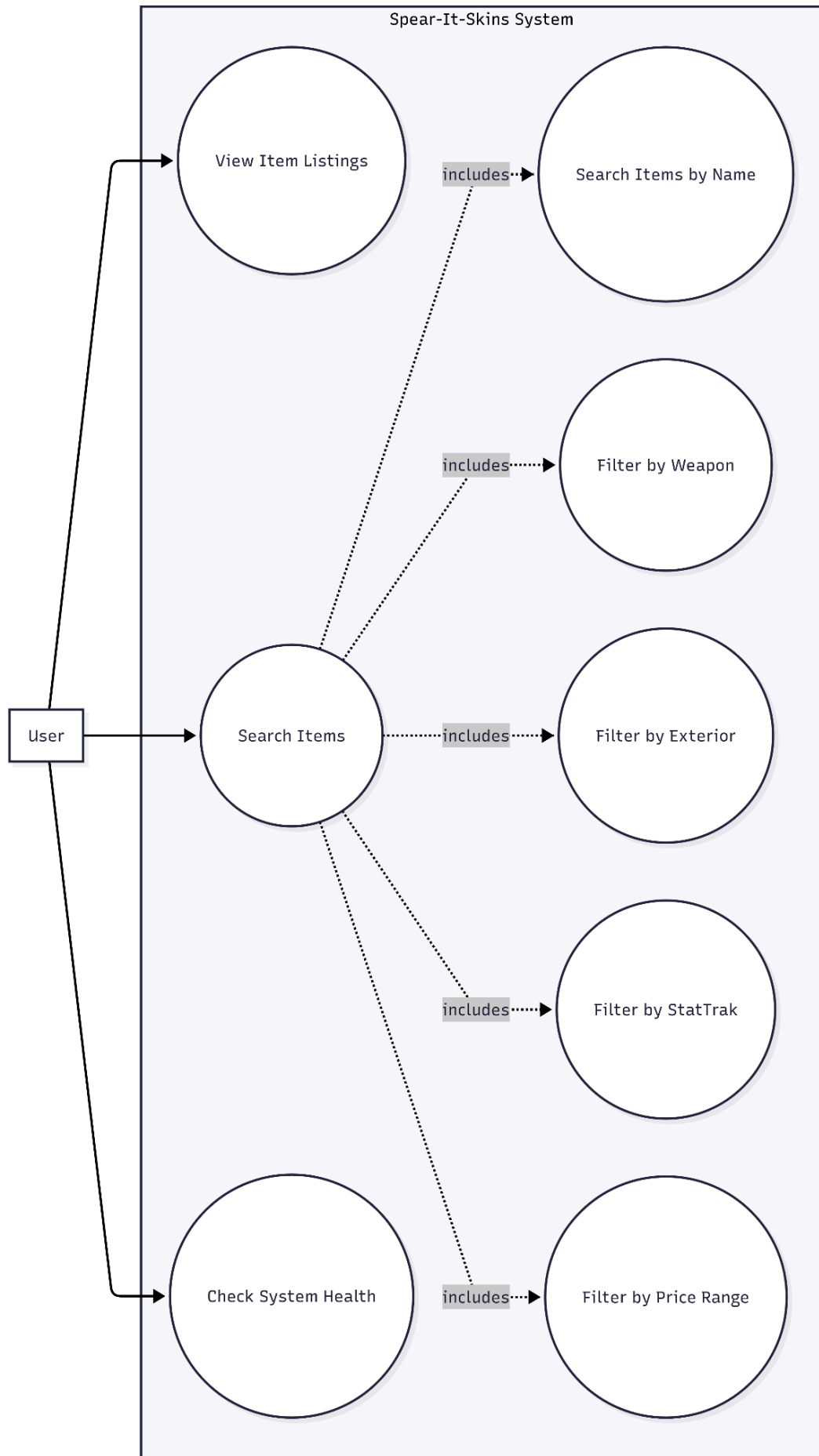


Figure 1. User-Facing Use Case Diagram

This diagram shows the user-facing functionality of the Spear-It-Skins system. The user can view CS2 item listings, search for items by name, filter results by weapon, exterior, StatTrak status, and price range, and check the system's health status. These use cases represent the primary ways a user interacts with the application through the frontend.

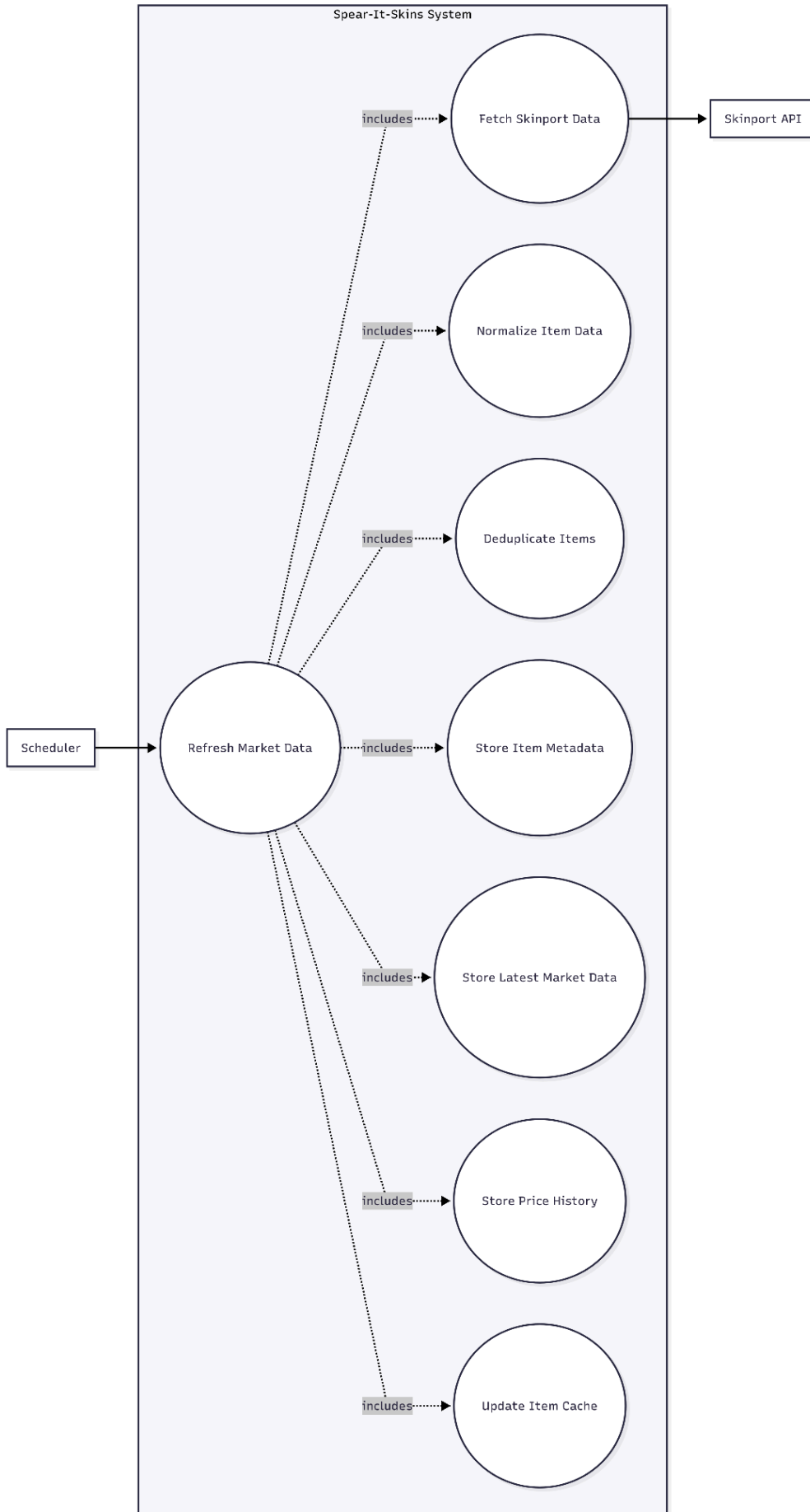


Figure 2. Backend Refresh Use Case Diagram

This diagram shows the backend market data refresh functionality of the Spear-It-Skins system. The scheduler triggers the refresh process, which fetches current item data from the Skinport API, normalizes and deduplicates the returned items, stores item metadata, latest market data, and historical price snapshots, and updates the item cache. These use cases represent the automated backend workflow that keeps item listings current.

4.1 Textual Use Case Descriptions

Use Case 1: View, Search, and Filter Items

Primary Actor: User

Description: The user views CS2 item listings and may search or filter results by item name, weapon, exterior condition, StatTrak status, or price range.

Preconditions: The backend service is running and item data is available in the system.

Main Flow:

1. The user opens the Spear-It-Skins application.
 2. The user views the available item listings.
 3. The user optionally enters a search term or selects one or more filters.
 4. The frontend sends the search and filter criteria to the backend items endpoint.
 5. The backend retrieves matching item data and returns a paginated response.
 6. The frontend displays the matching CS2 items to the user.
- Postconditions:** The user sees item listings that match the selected search and filter criteria.

Use Case 2: Check System Health

Primary Actor: User

Description: The user or frontend checks whether the backend service and database connection are available.

Preconditions: The backend service is reachable.

Main Flow:

1. The frontend sends a request to the health endpoint.
 2. The backend returns general service health information.
 3. The frontend may send a request to the database health endpoint.
 4. The backend checks whether database access is enabled and verifies the database connection.
 5. The backend returns the database status to the frontend.
- Postconditions:** The frontend receives information about whether the backend and database are available.

Use Case 3: Refresh Market Data

Primary Actor: Scheduler

Description: The scheduler triggers an automatic refresh of CS2 market data so item listings remain current.

Preconditions: The backend can access the Skinport API and the database connection is correctly configured.

Main Flow:

1. The scheduler starts the refresh process.
2. The backend checks whether another refresh is already running.
3. The backend fetches current item market data from the Skinport API.
4. The backend normalizes the returned item data into the system's internal format.

5. The backend removes duplicate logical items using the item's market hash name.
6. The backend stores item metadata, latest market data, and historical price snapshots.
7. The backend updates the item cache after a successful refresh.

Postconditions: The system has updated market data available for future user requests.

Use Case 4: Store Market Data

Primary Actor: Backend system

Description: The backend stores normalized item information in the database during the refresh process.

Preconditions: Normalized item data has been fetched from the Skinport API.

Main Flow:

1. The backend begins a database transaction.
2. The backend inserts or updates stable item metadata.
3. The backend inserts or updates the latest market state for each item.
4. The backend inserts a price history snapshot for each item.
5. The backend commits the transaction if all database operations succeed.
6. If an error occurs, the backend rolls back the transaction.

Postconditions: Item metadata, latest market data, and price history are safely stored without partial database updates.

5. Class Diagram and/or Sequence Diagrams (15 points)

This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.

If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram of the entire system and Sequence Diagrams for the three (3) most important use cases in your system**.

If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams, but for all the use cases of your system**. In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.

Class Diagrams show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.

A **Sequence Diagram** simply depicts **interaction between objects** (or **functions** - in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function.

The Spear-It-Skins backend is implemented primarily as a module-based Node.js/Express system rather than a traditional object-oriented application. Therefore, the following sequence diagrams use backend modules, route handlers, services, repository functions, and database components as lifelines. These diagrams show the order of interactions for the system's most important use cases.

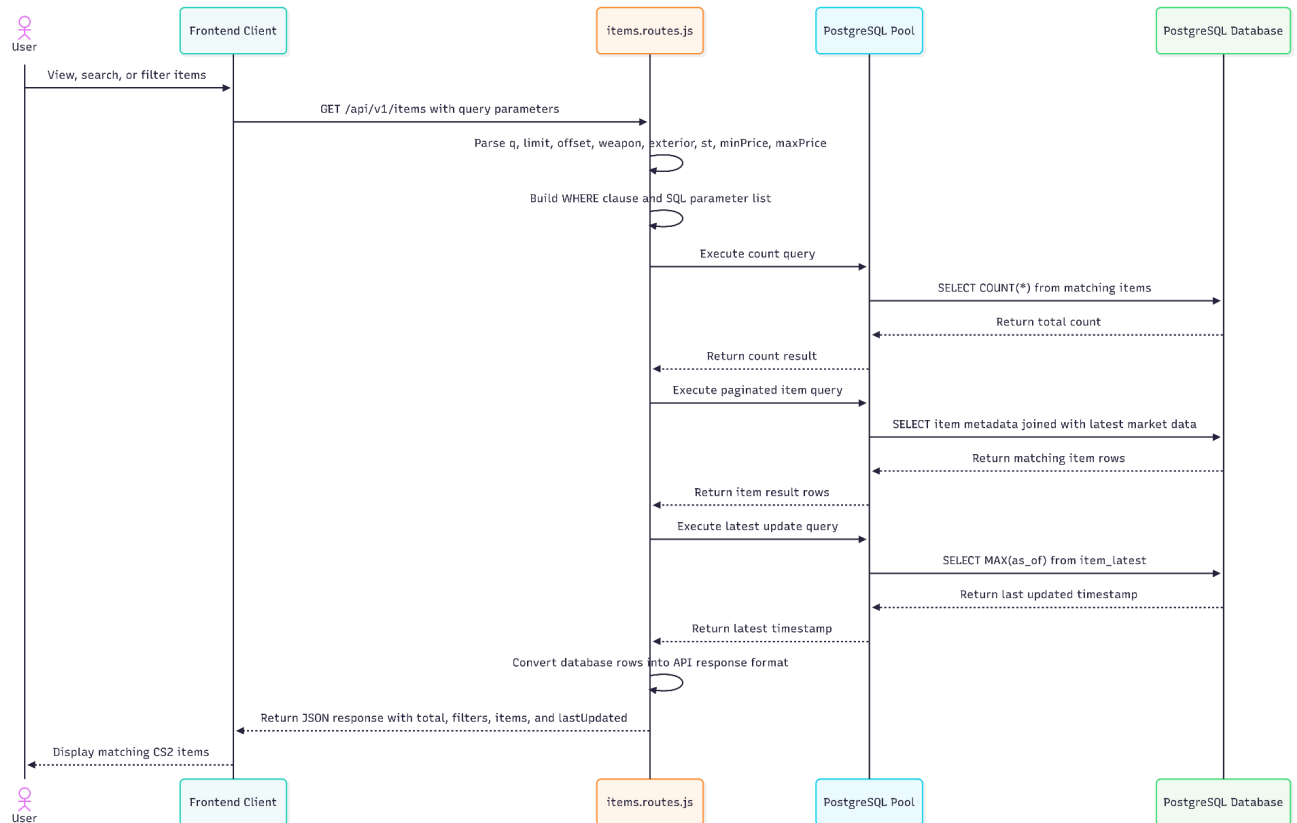


Figure 3. Search, Filter, and View Items Sequence Diagram

This sequence diagram shows how the system handles viewing, searching, and filtering item listings. The frontend sends query parameters to the backend items route. The backend parses the filters, queries PostgreSQL for the total count and paginated item results, formats the database rows into a JSON response, and returns the results to the frontend for display.

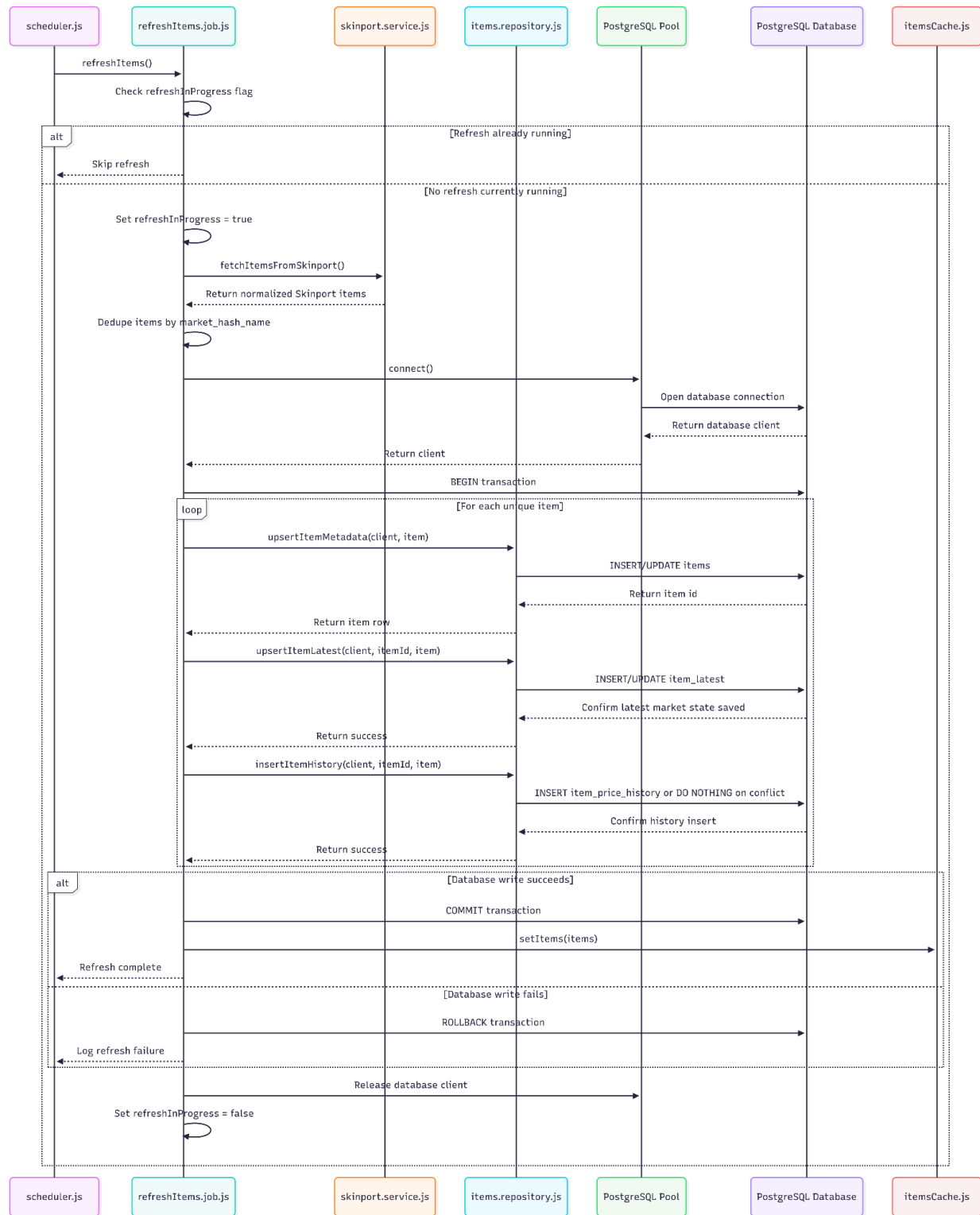


Figure 4. Scheduled Market Data Refresh Sequence Diagram

This sequence diagram shows the scheduled market refresh process. The scheduler triggers the refresh job, which prevents overlapping refreshes, fetches normalized item data from Skinport, deduplicates items by market hash name, and writes item metadata, latest market state, and historical price snapshots to PostgreSQL inside a transaction. If the database write succeeds, the transaction is committed and the cache is updated. If an error occurs, the transaction is rolled back.

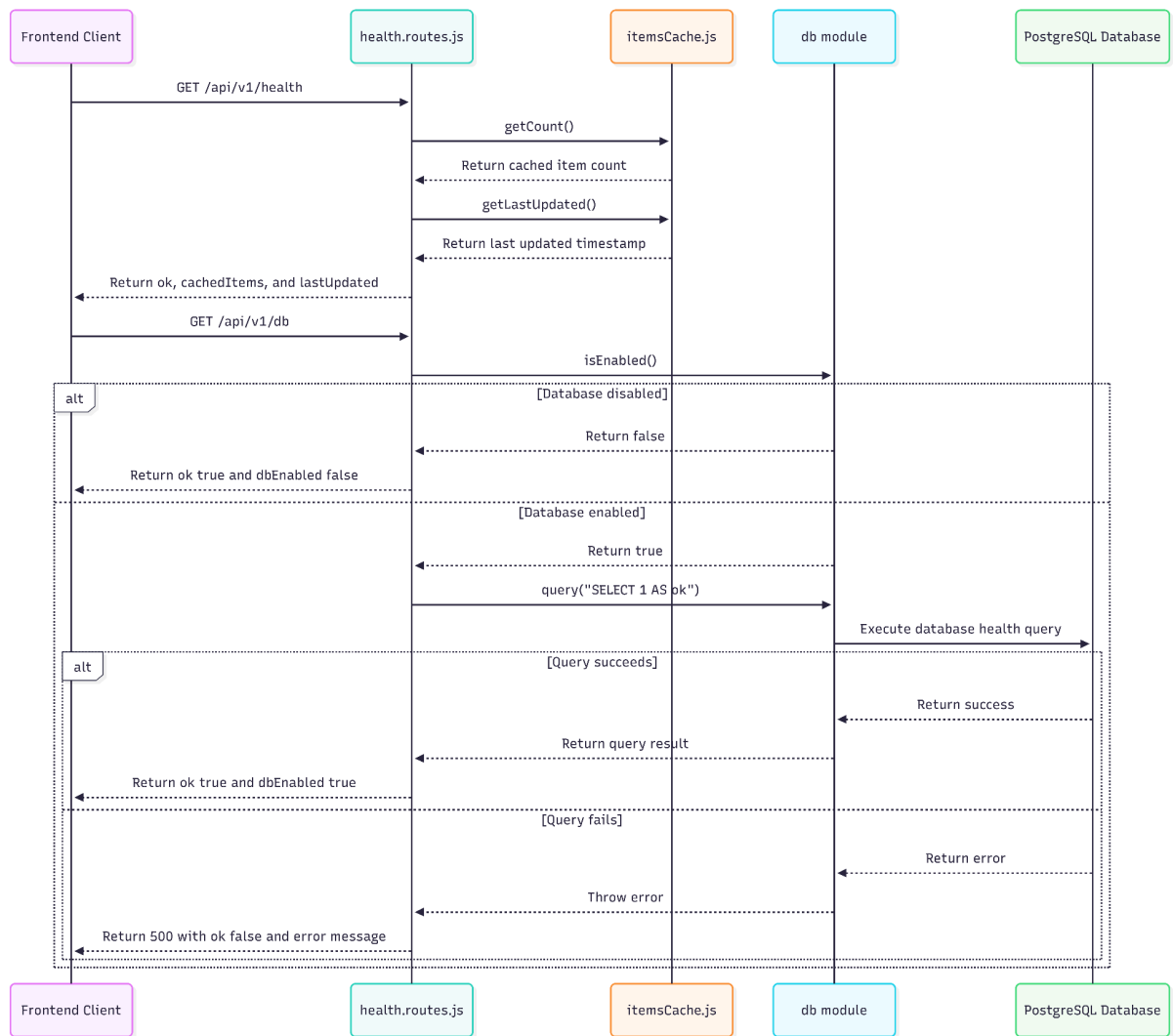


Figure 5. Health Check Sequence Diagram

This sequence diagram shows how the system handles health checks. The general health endpoint reports whether the backend is running and returns cache information. The database health endpoint checks whether database access is enabled and then runs a simple query to verify that PostgreSQL is reachable.

6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

The system operates as a Node.js backend application using Express. It is intended to run on a Linux development environment, local development machines, or a cloud hosting provider. The backend connects to a PostgreSQL database using the pg Node.js package. Environment variables are used to configure the server port and database connection string.

During development, the backend can be run with Node.js and npm. The system also supports local database development through PostgreSQL and can be tested using tools such as curl, jq, or a frontend web client.

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

Assumptions:

- The Skinport API remains available and returns item data in a usable JSON format.
- The database connection string is correctly configured through environment variables.
- The frontend expects item data in the format returned by the backend API.
- Market prices may change frequently, so displayed prices are only accurate as of the latest refresh.
- Some items may not have complete metadata or image URLs depending on the source API.
- The backend assumes market_hash_name can uniquely identify a logical item.

Dependencies: Node.js, Express, PostgreSQL, Skinport API, pg, node-cron, dotenv, cors, morgan, frontend client, (hosting provider)